Multi-Container Web App with Docker Compose

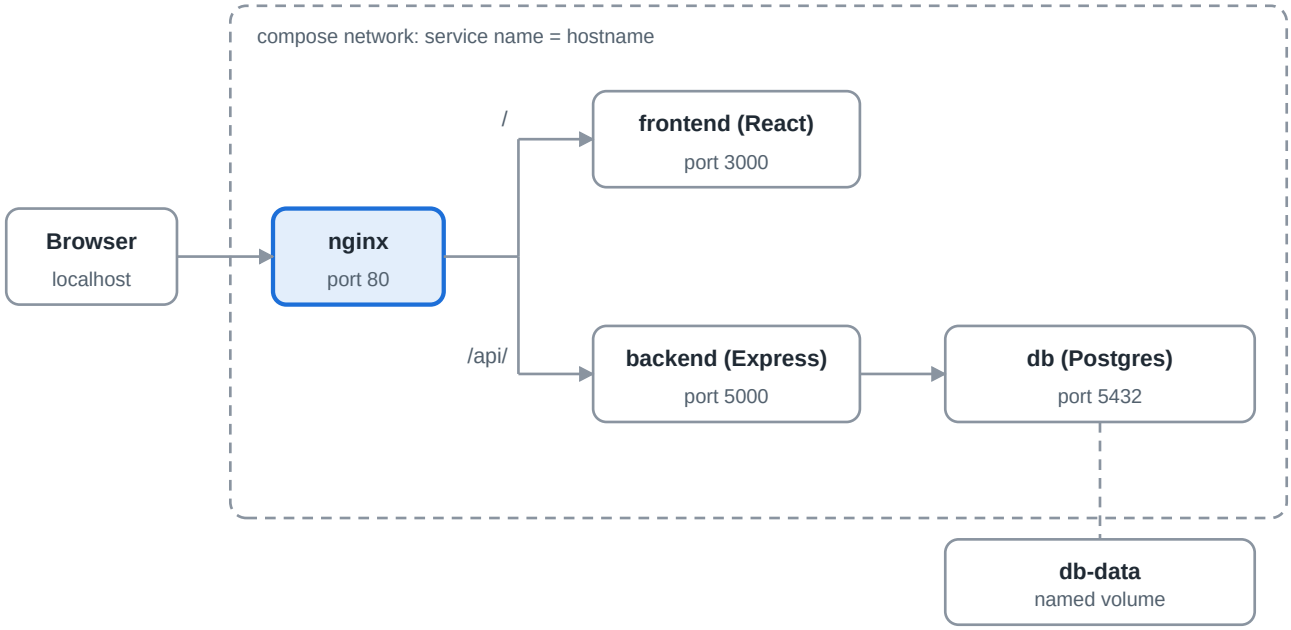
Step-by-step notes: React + Express + Postgres + Nginx | October 13, 2026

1. Project overview

This is a 4-container web app (React, Express, Postgres and Nginx) started with one `docker-compose.yml`. The goal of the project was to learn Docker hands-on.

- **Frontend:** React (Create React App), dev server on port `3000`
- **Backend:** Node/Express with the `pg` driver, port `5000`, routes `/api/health` and `/api/items` (GET, POST)
- **Database:** Postgres 16 (`postgres:16-alpine`), data kept in the named volume `db-data`
- **Reverse proxy:** Nginx on port `80`: `/` goes to the frontend, `/api/` goes to the backend
- **Who wrote what:** the React/Express/SQL app code was provided as a starter. The Dockerfiles, `docker-compose.yml` and `nginx/default.conf` were written by Karan himself.

Browser talks only to Nginx; everything else stays on the internal network



Browser talks only to nginx (port 80). Nginx sends `/` to the frontend and `/api/` to the backend, the backend connects to `db`, and the database data lives in the `db-data` volume.

Folder structure

```
Multi-Container Web App (Compose)/
|-- docker-compose.yml
|-- nginx/
|   `-- default.conf
|-- frontend/
|   |-- Dockerfile
|   |-- package.json
|   |-- public/index.html
|   `-- src/ (index.js, App.js)
|-- backend/
|   |-- Dockerfile
|   |-- package.json
|   |-- server.js
|   `-- .env.example
`-- db/
    `-- init.sql
```

- `db/init.sql` is not mounted in compose. The table is created by `CREATE TABLE IF NOT EXISTS` in `backend/server.js`.
- The Compose project name comes from the folder name, so renaming or moving the folder creates new images and a new volume.

2. Setup: Docker Desktop (Windows)

The Docker Desktop **app** must be open before anything works, because that is what starts the Docker daemon (the engine).

- Open **Docker Desktop** from the Start menu.
- Wait until the whale icon in the taskbar stops saying "starting" (30 to 60 seconds).
- Check the daemon:

```
docker version
```

- You should see both a **Client** and a **Server** section.
- If only Client shows and you get `failed to connect to the docker API at npipe:////./pipe/dockerDesktopLinuxEngine`, Docker Desktop is not running. This can happen after every PC restart.
- Optional: Settings > General > "Start Docker Desktop when you sign in" starts it automatically.

3. Step 1: Backend Dockerfile

Core concepts: an **image** is a read-only blueprint, a **container** is a running instance of an image, and a **Dockerfile** is the list of instructions that builds an image.

`backend/Dockerfile`:

```
FROM node:20-alpine
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
EXPOSE 5000
CMD ["node", "server.js"]
```

What each line does

- `FROM node:20-alpine`: the base image. The `alpine` variant is small.

- `WORKDIR /app`: the working folder inside the container; all later commands run there.
- `COPY package.json .`: copy only the dependency file first. Source `.` is the host build folder, destination `.` is the `WORKDIR`.
- `RUN npm install`: installs dependencies at build time.
- `COPY . .`: now copy the rest of the code.
- `EXPOSE 5000`: documents the port the app listens on (the default `PORT` in `server.js`).
- `CMD ["node", "server.js"]`: what runs when the container starts.

Why COPY twice? Layer caching

The real reason is **layer caching**, not image size. Docker builds one layer per instruction and reuses a layer if its input has not changed.

- Only `server.js` changed: the `npm install` layer comes from cache and the rebuild takes seconds.
- If `COPY . .` came first, every code change would re-run `npm install`.

Mistakes made on the way (and fixes)

- `COPY package.json` without a destination. COPY always needs a source and a destination.
- `COPY ..` is wrong (`..` means one folder up, outside the build context). Use `COPY . .`
- `EXPOSE port:5000` is wrong. Just the number: `EXPOSE 5000`.
- `CMD ["node"],["server.js"]` is wrong. CMD takes one array: `CMD ["node", "server.js"]`.

4. Step 2: Build and run the backend

Run from inside the `backend/` folder (where the Dockerfile is):

```
cd backend
docker build -t my-backend .
docker images
docker run -p 5000:5000 my-backend
```

- `-t my-backend`: gives the image a name (tag).
- The final `.` is the **build context**: "take files from this folder".
- `-p 5000:5000` is `host_port:container_port`.
- In the build output every Dockerfile line is a step (`[3/5] COPY package.json .`); unchanged steps show `CACHED`.

Test it in the browser: `http://localhost:5000/api/health` returns `{"status":"ok","service":"backend"}`.

Foreground vs -d

- Without `-d` the container holds your terminal; stop it with `Ctrl+C`.
- With `-d` (detached) it runs in the background and the terminal stays free.
- At this stage `DB init failed` in the logs was normal because Postgres did not exist yet.

Useful commands

```
docker ps            # running containers
docker ps -a         # including stopped ones
docker logs <name>   # container logs
docker stop <name>
docker rm <name>
```

5. Step 3: Postgres, network and DATABASE_URL

Postgres needs no Dockerfile. Docker Hub has an **official image**, so a plain `docker run` is enough (a backtick at the end of a line is the PowerShell line continuation):

```
docker run --name my-postgres `
  -e POSTGRES_USER=appuser -e POSTGRES_PASSWORD=apppass -e POSTGRES_DB=appdb `
  -p 5432:5432 -d postgres:16-alpine
```

- `--name` is the **container** name (a nickname). The last item, `postgres:16-alpine`, is the **image** name. They are different things.
- `-e KEY=value` sets an environment variable. The Postgres image reads these on first start to create the user, password and database.

Problem: containers cannot find each other

- Inside a container, `localhost` means that container itself, not your Windows machine and not another container.
- So the backend's `localhost:5432` never reaches Postgres.

Fix: custom network + container name as hostname

```
docker network create my-app-network

docker run --name my-postgres --network my-app-network `
  -e POSTGRES_USER=appuser -e POSTGRES_PASSWORD=apppass -e POSTGRES_DB=appdb `
  -p 5432:5432 -d postgres:16-alpine

docker run --name my-backend --network my-app-network -p 5000:5000 `
  -e DATABASE_URL=postgres://appuser:apppass@my-postgres:5432/appdb `
  -d my-backend
```

- `--network my-app-network` puts both containers on the same network.
- On the same network, Docker's internal **DNS** turns a container name into its IP, so the URL uses `my-postgres` as host.
- `-e` only applies when the container is **created**. To change an env var, `stop` + `rm` the container and `run` it again.

DATABASE_URL format

```
postgres://USERNAME:PASSWORD@HOST:PORT/DBNAME
```

- It starts with `postgres://`, not `https://`.
- `@HOST:PORT` is always required.

End-to-end test

```
docker logs my-backend
curl.exe -X POST http://localhost:5000/api/items `
  -H "Content-Type: application/json" -d '{\"name\":\"first item\"}'
```

- `docker logs my-backend` should show only `Backend listening on port 5000`, with no `DB init failed`.
- In PowerShell, `curl` is an alias for `Invoke-WebRequest`, so write `curl.exe` (or use `Invoke-WebRequest -Method POST ...`).
- `http://localhost:5000/api/items` returned `[]`, and after the POST it showed the item. The data is saved in the Postgres container.

6. Step 4: Frontend Dockerfile

Same structure as the backend. Only the port and the `CMD` differ.

`frontend/Dockerfile`:

```
FROM node:20-alpine
WORKDIR /web
COPY package.json .
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

- `EXPOSE 3000` is the default port of the React dev server.
- Use `CMD ["npm", "start"]`, not `CMD ["react-scripts","start"]`. The `react-scripts` binary lives in `node_modules/.bin/`, which is not on the container's PATH; `npm` finds it by itself.
- `npm start` runs whatever `scripts.start` says in `package.json` (`react-scripts start`).
- An empty `FROM` line makes the build fail.

Build and run

```
cd frontend
docker build -t my-frontend .
docker run --name my-frontend --network my-app-network -p 3000:3000 -d my-frontend
```

- Always run `docker build` from the folder that holds the Dockerfile. From the wrong folder you get `Cannot find path`.
- The image name goes **last** in the `docker run` command.

The proxy field (where the backend URL is set)

The `docker run` command contains no backend URL. React's `fetch("/api/items")` is forwarded by the CRA dev server using the `proxy` field in `package.json`:

```
"proxy": "http://backend:5000"
```

- Without Docker, `http://localhost:5000` works.
- Inside a container `localhost` is the frontend itself, so the backend's **name** is needed.
- With manual `docker run` the name was `my-backend`; in Compose the service name is `backend`, so `http://backend:5000`.
- After editing `package.json`, **rebuild** the image.

Why is the frontend image over 1 GB?

- `react-scripts` brings webpack, babel, eslint, jest and many other dev tools.
- The container runs the dev server (`npm start`), not a production build, so all of that stays in the image.
- Backend image: about 221 MB. Frontend image: about 1.18 GB.
- In production you run `npm run build` and serve static files from Nginx (a **multi-stage build**), which can get the image to about 25 MB. This is still on the to-do list.

7. Step 5: docker-compose.yml

Three `docker run` commands, the network and the env vars are now described in one YAML file, and `docker compose up` starts everything together. The file goes in the **project root** (next to `frontend/` and `backend/`), not inside them.

What Compose automates

- It creates a **network** for you (`<folder>_default`), so no `docker network create`.
- Every service name becomes a hostname on that network (`db`, `backend`, `frontend`).
- Container names are generated (`<folder>-db-1`); no `--name` or `-d` needed.

image vs build

- **Ready-made image** (Postgres, Nginx): `image: postgres:16-alpine`.
- **Your own Dockerfile** (frontend, backend): `build: ./backend`. The value is a **folder path** where the Dockerfile lives, not a command like `npm install`.
- Do not use both in one service. `FROM node:20-alpine` is already inside the Dockerfile.

First working version (db + frontend + backend)

```
services:
  db:
    image: postgres:16-alpine
    ports:
      - "5432:5432"
    environment:
      - POSTGRES_USER=appuser
      - POSTGRES_PASSWORD=apppass
      - POSTGRES_DB=appdb

  frontend:
    ports:
      - "3000:3000"
    build: ./frontend

  backend:
    ports:
      - "5000:5000"
    build: ./backend
    environment:
      - DATABASE_URL=postgres://appuser:apppass@db:5432/appdb
```

- The host in `DATABASE_URL` is `db` because the Postgres service is named `db` in the compose file.
- The frontend `proxy` in `package.json` must be `http://backend:5000`.

Running it

```
cd <project root>
docker compose up --build
```

- Before the first `docker compose up`, `stop` and `rm` the old manual containers (`my-backend`, `my-postgres`, `my-frontend`). Otherwise: `Bind for 0.0.0.0:5000 failed: port is already allocated`.
- Always run it from the project root, not from inside `frontend/` or `backend/`.
- `Ctrl+C` stops it. Use `docker compose up -d` to run in the background.

8. Step 6: Healthcheck and depends_on (startup race)

The backend must start only when Postgres is **really** accepting connections, not just when its container has started.

Symptoms seen

- `DB init failed: getaddrinfo EAI_AGAIN db`: the backend started before Postgres, so the name did not resolve yet.
- `DB init failed: connect ECONNREFUSED 172.19.0.2:5432`: the name resolved, but Postgres was not listening on TCP yet.

Fix 1: depends_on with service_healthy

Plain `depends_on: [db]` only waits until the container has **started**. With a condition, the backend waits until `db` is healthy:

```
backend:
  depends_on:
    db:
      condition: service_healthy
```

Fix 2: a healthcheck on db

```
db:
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -h 127.0.0.1 -U appuser -d appdb"]
    interval: 5s
    timeout: 5s
    retries: 5
```

- `pg_isready` asks Postgres "are you ready?" every 5 seconds, up to 5 tries.

Fix 3: why -h 127.0.0.1 matters

- On first start Postgres runs **twice**: first a temporary instance (Unix socket only, for setup), then it shuts down and the real instance starts (on TCP).
- Without `-h`, `pg_isready` reports "healthy" even for the temporary instance.
- With `-h 127.0.0.1` the check goes over TCP, so only the real instance counts as healthy.
- In the logs, `Container ...-db-1 Healthy` now appears before `Backend listening on port 5000`, and `DB init failed` is gone.

9. Step 7: Volumes (data persistence)

The problem, tested first

- Added an item, then ran `docker compose down` and `docker compose up`.
- `http://localhost:5000/api/items` returned `[]`: the data was gone.
- The logs showed Postgres initializing from scratch (`running bootstrap script`, `creating configuration files`).
- The data lived **inside** the container, and `down` deletes the container.

The fix: a named volume

Two places must change, and both are required:

```
services:
  db:
    # ...
    volumes:
      - db-data:/var/lib/postgresql/data   # 1. mount it in the db service

volumes:
  db-data:                                 # 2. declare it at the end of the file
                                           #    (same level as "services:")
```

- `/var/lib/postgresql/data` is Postgres's data folder inside the container.
- `db-data` is managed by Docker and lives **outside** the container.
- If you forget the declaration, Docker does not know it should create `db-data`.

Test that passed

1. `docker compose down`, then `docker compose up`.
2. POST an item (`volume test`).
3. `docker compose down`, then `docker compose up` again.
4. `http://localhost:5000/api/items` still showed the item.

When is the volume deleted?

```
docker compose down       # removes containers, keeps the volume
docker compose down -v    # also removes volumes (data is deleted for good)
```

Bind mount vs named volume

	Named volume	Bind mount
Example	`db-data:/var/lib/postgresql/data`	`./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro`
Left side	A name Docker manages	A path on your host
Where the data lives	Managed by Docker	In your own folder
Use it for	Database data	Config files, source code

Reminder: the Compose project name comes from the folder name. If the folder changes, a new volume is created and the old data does not show up.

10. Step 8: Nginx reverse proxy

Why

- Before, the frontend was on `:3000` and the backend on `:5000`, both exposed separately.
- In a real setup only **one entry point** (port 80/443) is public and everything else stays internal.
- Nginx sends `/` to the frontend and `/api/` to the backend. The browser sees a single origin (`localhost`), so there is no CORS trouble either.

nginx/default.conf

```
server {
    listen 80;

    location / {
        proxy_pass http://frontend:3000;
        proxy_set_header Host $host;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }

    location /api/ {
        proxy_pass http://backend:5000;
        proxy_set_header Host $host;
    }
}
```

- `proxy_pass http://frontend:3000`: the service **name** plus the port **inside** the container.
- Do **not** put a `/` at the end of `proxy_pass http://backend:5000;`. The backend routes already start with `/api/...`; a trailing slash makes nginx strip the `/api` prefix and you get 404.
- The `Upgrade` and `Connection` lines are for the React dev server's **websocket** (hot reload).
- `proxy_set_header Host $host` fixes the "Invalid Host header" error below.

The nginx service in compose

```
  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - frontend
      - backend
```

- `./nginx/default.conf` is the file on your PC.
- `/etc/nginx/conf.d/default.conf` is where nginx reads its config inside the container (fixed by the image).
- `:ro` means read-only; the container cannot modify the file.
- This is a **bind mount**: nothing is copied, both sides are the same file, and it overrides nginx's default config.
- After editing the config you do not rebuild; just run `docker compose restart nginx`.
- In YAML, `nginx:` must be at the **same indentation** as `db:`, `frontend:` and `backend:`. Indent it more and YAML treats it as part of `backend`.

Error: "Invalid Host header"

- Opening `http://localhost` showed this text.
- By default nginx sends `Host: frontend:3000` upstream.
- The React dev server (webpack) checks the Host header for security and rejects `frontend` as unknown.
- Fix: `proxy_set_header Host $host;` (pass the browser's original host), then:

```
docker compose restart nginx
```

Test (3 URLs)

- `http://localhost`: the React app.
- `http://localhost/api/health`: `{"status":"ok","service":"backend"}`.
- `http://localhost/api/items`: the list of items.
- If port 80 is busy (IIS, Skype), use `"8080:80"` in compose and open `http://localhost:8080`.

11. Errors we hit and their fixes

If something breaks, look here first.

Error / symptom	Cause	Fix
`failed to connect to the docker API at npipe:////./pipe/dockerDesktopLinuxEngine`	Docker Desktop app is closed, so the daemon is not running	Open Docker Desktop; run `docker version` and wait for the Server section
`Cannot find path ...\frontend`	`cd` / `docker build` run from the wrong folder	Go to the project folder; check with `dir` that the Dockerfile is there
Dockerfile: empty `FROM` / `COPY` without destination	Syntax: COPY needs source and destination	`COPY package.json .` and `FROM node:20-alpine`
`DB init failed` (manual containers)	Inside a container `localhost` is the container itself	Custom network and the container/service name in `DATABASE_URL`
PowerShell: `curl ... Cannot bind parameter 'Headers'`	In PowerShell `curl` is an alias of `Invoke-WebRequest`	Use `curl.exe ...` or `Invoke-WebRequest -Method POST ...`
`Bind for 0.0.0.0:5000 failed: port is already allocated`	Old manual containers still hold the port	`docker stop` + `docker rm` the old containers, then `docker compose up`
`DB init failed: getaddrinfo EAI_AGAIN db`	Backend started before Postgres	`depends_on` with `condition: service_healthy` plus a db healthcheck
`DB init failed: connect ECONNREFUSED`	Healthcheck passed on the temporary Postgres instance	`pg_isready -h 127.0.0.1 ...`
Data gone after `down`	Data was stored inside the container	Named volume `db-data` (mount it and declare it at the top level)
`Invalid Host header`	Nginx sent `Host: frontend:3000` and webpack rejected it	`proxy_set_header Host $host;` then `docker compose restart nginx`
New images/volume after moving the folder, old data missing	Compose project name is taken from the folder name	Expected. For the old data, run Compose from the old folder name

12. Final files, cheat sheet and next steps

Final docker-compose.yml (what is running now)

```
services:
  db:
    image: postgres:16-alpine
    ports:
      - "5432:5432"
    environment:
      - POSTGRES_USER=appuser
      - POSTGRES_PASSWORD=apppass
      - POSTGRES_DB=appdb
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -h 127.0.0.1 -U appuser -d appdb"]
      interval: 5s
      timeout: 5s
      retries: 5
    volumes:
      - db-data:/var/lib/postgresql/data

  frontend:
    ports:
      - "3000:3000"
    build: ./frontend

  backend:
    ports:
      - "5000:5000"
    build: ./backend
    environment:
      - DATABASE_URL=postgres://appuser:apppass@db:5432/appdb
    depends_on:
      db:
        condition: service_healthy

  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - frontend
      - backend

volumes:
  db-data:
```

Command cheat sheet

```
docker compose up              # start (no build if images already exist)
docker compose up --build      # when code / Dockerfile / package.json changed
docker compose up -d           # in the background
docker compose down            # remove containers + network, keep the volume
docker compose down -v         # also remove the volume (deletes DB data)
docker compose restart nginx   # only a bind-mounted config changed
docker compose logs nginx      # logs of one service
docker compose ps              # containers of this compose project
docker ps -a                   # all containers
docker images                  # all images
```

- `--build` is needed when something **baked into the image** changed: the Dockerfile, `package.json`, or code copied by `COPY . .`
- If only a bind-mounted config changed, `restart` is enough; no rebuild.

- Adding `--build` when nothing changed is harmless; unchanged steps show `CACHED`.

Next steps

- **Remaining cleanup:** remove the `ports:` of `frontend` and `backend` (and `db`, if you do not need a DB client). Containers keep talking over the internal network, and only nginx (port 80) is reachable from outside. Then run `docker compose up -d` and check that `:3000` and `:5000` no longer open.
- **Secrets:** the DB password is hardcoded in compose. Move it to a `.env` file and use `env_file:`.
- **Production frontend:** `npm run build` plus a **multi-stage Dockerfile**, with static files served by Nginx (image from about 1.18 GB to about 25 MB).
- **Registry:** push the images to Azure Container Registry.